

Contexto de las decisiones

Solo vos, fullstack, sólido en el stack, 6 meses para el MVP, multi-tenant desde el día uno, web responsiva ahora, app nativa en V2, doble moneda. Eso define un perfil claro: las decisiones tienen que minimizar la superficie de código que mantenés solo, maximizar lo que el stack hace por vos, y no cerrar puertas para V2.

Cada decisión de arquitectura que sigue tiene una justificación. No es "usá esto porque es moderno" — es "usá esto porque dado tu contexto es la opción que menos te va a costar mantener solo durante 6 meses".

Stack tecnológico

Frontend: React + TypeScript + Vite

React porque ya lo conocés y porque el ecosistema de componentes para dashboards de datos es el más maduro. TypeScript es no negociable en un sistema con un modelo de dominio complejo — los tipos te van a salvar de bugs que en JavaScript solo aparecen en producción. Vite porque el tiempo de arranque importa cuando trabajás solo y necesitás ciclos rápidos.

Para UI components: shadcn/ui. No es una librería de componentes tradicional — es un conjunto de componentes que copiás a tu proyecto y controlás completamente. Esto importa porque el diseño de un SaaS ganadero tiene que ser funcional y denso en información, no un template genérico. Para tablas de datos grandes: TanStack Table. Para gráficos de peso y GDP: Recharts. Para manejo de estado del servidor: TanStack Query. Para formularios: React Hook Form + Zod.

Lo que no usás: Redux ni ningún gestor de estado global complejo. TanStack Query maneja el estado del servidor. Para estado local de UI, useState y useContext alcanzan.

Backend: FastAPI + Python

Ya lo conocés. FastAPI con Pydantic v2 para validación de datos te da la capa de reglas de negocio del servidor bien separada del modelo de datos. Alembic para migraciones de base de datos — en un sistema con un modelo tan definido desde el

principio, las migraciones van a ser frecuentes durante el desarrollo y necesitás control total sobre ellas.

Una decisión importante: no uses las funciones de base de datos de Supabase directamente desde el frontend para nada que tenga lógica de negocio. Todo pasa por FastAPI. Supabase es tu base de datos PostgreSQL gestionada y tu sistema de autenticación — no tu backend. Esta separación es lo que te permite mantener las reglas de negocio (RN-01 a RN-25) en un solo lugar y testearlas.

Base de datos: Supabase (PostgreSQL)

PostgreSQL es la elección correcta para este modelo de dominio por tres razones concretas. Primero, los datos temporales y los cálculos históricos (existencia en fecha pasada, recálculo de rodeo) se resuelven con queries SQL que en una base documental serían pesadillas. Segundo, las relaciones entre entidades son ricas y el motor relacional las maneja con integridad referencial real. Tercero, cuando llegue el Centro de Decisiones analítico, vas a querer correr agregaciones sobre millones de eventos — PostgreSQL escala para eso.

Supabase te da el hosting gestionado, autenticación con Row Level Security, storage para archivos (PDFs de guías, CSVs de importación) y una API de realtime que vas a necesitar para las alertas en V2. El plan gratuito alcanza para desarrollo y clientes beta. El plan Pro (25 USD/mes) cuando tengas los primeros clientes pagos.

Autenticación: Supabase Auth

Ya está integrada con tu base de datos. Maneja JWT, refresh tokens, OAuth si lo necesitás después. Lo importante: vas a usar Row Level Security (RLS) de PostgreSQL para el aislamiento multi-tenant. Cada tabla tiene una política que filtra por `establecimiento_id` basándose en el JWT del usuario. Esto significa que aunque haya un bug en el backend, la base de datos nunca devuelve datos de un establecimiento a un usuario de otro. Es la capa de seguridad más profunda que podés tener.

Hosting: Firebase Hosting + Cloud Run

Firebase Hosting para el frontend estático — deploy en segundos, CDN global, SSL automático. Cloud Run para el backend FastAPI en contenedor — escala a cero cuando no hay tráfico (importante para costos en las primeras etapas), escala automáticamente cuando hay carga, sin gestión de servidores.

Esta combinación tiene un costo muy bajo al principio: con pocos clientes pagás casi nada en Cloud Run porque el servicio está dormido la mayor parte del tiempo. Cuando crezcás, el costo crece linealmente con el uso.

Estructura del proyecto

Un monorepo con separación clara entre frontend y backend. No un monorepo con herramientas complejas como Turborepo — simplemente una carpeta raíz con dos subdirectorios y un README que explica cómo arrancar cada uno.

```
/
├── frontend/
│   ├── src/
│   │   ├── features/    # arquitectura por features
│   │   │   ├── animales/
│   │   │   ├── lotes/
│   │   │   ├── potreros/
│   │   │   ├── movimientos/
│   │   │   ├── sanidad/
│   │   │   ├── pesajes/
│   │   │   ├── reportes/
│   │   │   └── dashboard/
│   │   ├── components/  # componentes reutilizables
│   │   ├── hooks/       # hooks globales
│   │   ├── services/    # llamadas a la API
│   │   ├── layouts/     # layouts de página
│   │   ├── lib/         # utilidades, config
│   │   └── styles/      # tokens, variables globales
│   └── ...
├── backend/
└── app/
```

```
| | ├── core/      # config, seguridad, db session
| | ├── db/       # base: engine, base model
| | ├── models/   # modelos SQLAlchemy
| | ├── schemas/  # esquemas Pydantic
| | ├── crud/     # operaciones de base de datos
| | ├── services/ # lógica de negocio
| | ├── routers/  # endpoints FastAPI
| | └── main.py
| ├── alembic/    # migraciones
| └── tests/
└── docs/         # documentación funcional
    ├── glosario.md
    ├── modelo-dominio.md
    ├── mvp.md
    ├── casos-de-uso.md
    ├── reglas-de-negocio.md
    └── indicadores.md
└── docker-compose.yml # entorno local
```

La carpeta docs es donde viven los documentos que acabás de escribir. No en Notion, no en Google Docs — en el repo. Cuando un desarrollador clona el proyecto, la documentación funcional está ahí. Cuando hagás un cambio de dominio, el commit del código y el commit del documento van juntos.

Modelo de datos — decisiones clave

El esquema completo de la base de datos lo desarrollamos en el próximo paso, pero acá están las decisiones de diseño que afectan todo lo demás.

Multi-tenant con Row Level Security

Todas las tablas de dominio tienen una columna `establecimiento_id` que referencia a la tabla `establecimientos`. Las políticas RLS de Supabase filtran automáticamente cada query por el establecimiento del usuario autenticado. El backend además valida el `establecimiento_id` en cada operación — doble capa de seguridad.

sql

-- Ejemplo de política RLS para la tabla `animales`

`create policy "usuarios ven solo animales de su establecimiento"`

`on animales for all`

`using (`

`establecimiento_id = (`

`select establecimiento_id`

`from usuarios_establecimientos`

`where user_id = auth.uid()`

`)`

`);`

Eventos como tabla central

Todos los eventos del sistema — pesajes, movimientos, vacunaciones, tratamientos — heredan de una tabla base eventos con los campos comunes (`id`, `fecha_evento`, `fecha_registro`, `usuario_id`, `establecimiento_id`, `observaciones`, `corregido_por`). Las especializaciones son tablas separadas que referencian a eventos con una relación uno a uno.

Esto implementa directamente RN-15 (inmutabilidad) y RN-16 (doble fecha) a nivel de esquema. No es posible editar un evento porque ningún endpoint de la API lo permite, y la tabla tiene un trigger que rechaza updates.

Historial de categorías como tabla separada

La categoría actual del animal no se almacena solo en la tabla `animales` — se almacena en una tabla `animal_categorias` con fecha de inicio y fecha de fin. La categoría actual es la que tiene `fecha_fin` nula. Esto implementa RN-05 directamente y hace posible IND-09 (existencia histórica con categoría correcta).

Moneda con tipo de cambio de referencia

Los eventos económicos almacenan monto y moneda (PYG o USD). El establecimiento puede configurar una tasa de conversión de referencia. Los indicadores económicos se calculan en la moneda original y se convierten solo para visualización consolidada. Nunca se almacena un valor convertido — solo los valores originales y la tasa de conversión usada al momento del reporte.

Soft delete solo para entidades de configuración

Los animales nunca se eliminan (RN-01). Las otras entidades de configuración — potreros, lotes — tampoco se eliminan físicamente, tienen un campo estado que puede ser activo, inactivo o archivado. Solo los usuarios administradores pueden archivar entidades, y archivar no es lo mismo que borrar.

Separación de responsabilidades en el backend

La regla es simple: cada capa hace una sola cosa.

routers reciben el request HTTP, validan que el usuario tenga permiso para la operación, llaman al servicio correspondiente y devuelven la respuesta. No tienen lógica de negocio ni queries de base de datos.

services implementan las reglas de negocio. Aquí viven RN-01 a RN-25. Un servicio puede llamar a múltiples operaciones de crud en una transacción atómica. Por ejemplo, el servicio de traslado interno valida la carga del potrero destino (RN-13), registra el movimiento por cada animal (RN-17), actualiza la asignación de potrero y recalcula las alertas (RN-14). Todo en una transacción — si algo falla, nada se guarda.

crud son funciones puras que hablan con la base de datos. Reciben un objeto de datos y una sesión de base de datos, ejecutan la query y devuelven el resultado. No validan reglas de negocio.

schemas son los contratos de la API. Definen qué campos acepta cada endpoint y qué devuelve. Pydantic valida automáticamente los tipos y formatos.

models son los modelos SQLAlchemy que mapean las tablas de PostgreSQL.

Decisiones sobre la API

API REST con versionado desde el principio: todos los endpoints bajo `/api/v1/`. Cuando en V2 necesites cambiar el contrato de algún endpoint sin romper clientes existentes, tenés `/api/v2/` disponible.

Autenticación con Bearer token JWT en todos los endpoints. El token lo emite Supabase Auth y el backend lo valida con la clave pública de Supabase.

Paginación con cursor en todas las listas que pueden crecer (animales, eventos, movimientos). Un establecimiento con 2000 animales no puede recibir todos de una sola query.

Los endpoints de escritura que afectan múltiples entidades (vacunación de lote, traslado masivo) son síncronos en el MVP — el cliente espera hasta que se procesen todos los animales. En V2, con lotes de más de 500 animales, esto migra a un job asíncrono con polling de estado. El diseño del endpoint debe contemplar esa migración: ya en el MVP el response incluye un campo `job_id` aunque en V1 siempre sea nulo.

Lo que no hacés en el MVP por razones técnicas correctas

No implementás caché de Redis ahora. TanStack Query maneja el caché del cliente, y PostgreSQL con índices correctos es suficientemente rápido para los volúmenes del MVP. Redis agrega operaciones, agrega costos y agrega un componente más que mantenés solo.

No implementás WebSockets para alertas en tiempo real. Las alertas se calculan cuando el usuario abre el dashboard (como define el MVP). En V2 esto migra a Supabase Realtime que ya está disponible en tu stack sin agregar infraestructura.

No implementás un sistema de colas para procesamiento asíncrono. Cloud Run + FastAPI síncronos es suficiente para el MVP. Si el procesamiento de una importación CSV de 1000 animales tarda 8 segundos, es aceptable. No es aceptable en V2 con 10.000 animales — pero ese problema lo resolvés en V2.

No implementás observabilidad completa desde el principio. Logs estructurados en Cloud Run con Google Cloud Logging es suficiente para el MVP. Datadog, Sentry o similares cuando tengas clientes en producción.

Deuda técnica que aceptás conscientemente

La misma que identificamos en el documento del MVP, ahora con su impacto técnico preciso.

El soporte de un solo establecimiento por usuario en la interfaz no afecta el modelo de datos (establecimiento_id está en todas las tablas desde el día uno) pero sí afecta la autenticación: en el MVP un usuario pertenece a un solo establecimiento. En V2 la tabla usuarios_establecimientos ya soporta la relación muchos a muchos — solo hay que exponer el selector en la UI y ajustar cómo se setea el establecimiento_id en el JWT.

Las alertas se calculan en el request. La función de cálculo de alertas está en el servicio, no en el router. Cuando en V2 migrés a un job que las precalcula, moverás esa función de servicio a un worker sin tocar la lógica.

Los reportes son queries directas a PostgreSQL que se convierten a PDF o Excel en el servidor. En V2 con más variedad de reportes, esto migra a una capa de reporting con plantillas. El diseño actual debe evitar lógica de presentación en las queries — la query devuelve datos, el formatter genera el archivo.